

Module2 Part 2

Uniform-cost Search Algorithm

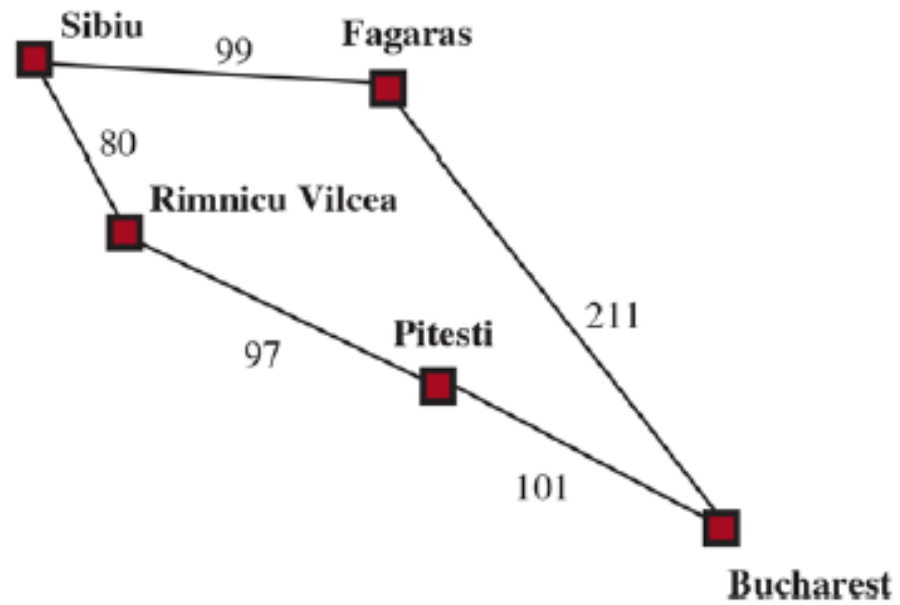
- **When actions have different costs**, an obvious choice is to use best-first search where the evaluation function is the cost of the path from the root to the current node.
- This is called Dijkstra's algorithm by the theoretical computer science community, and **uniform-cost search** by the AI community.
 - Breadth-first search spreads out in waves of uniform depth—first depth 1, then depth 2, and so on
 - uniform-cost search spreads out in waves of uniform path-cost.

Uniform-cost Search Algorithm

- Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph.
- This algorithm shall be used when a different cost is available for each edge.
- Goal: To find a path to the goal node which has the lowest cumulative cost.
- It expands nodes according to their path costs from the root node.
- It can be used to solve any graph/tree where the optimal cost is in demand.
- A uniform-cost search algorithm is implemented by the priority queue.
- It gives maximum priority to the lowest cumulative cost.
- Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

Uniform-cost Search Algorithm

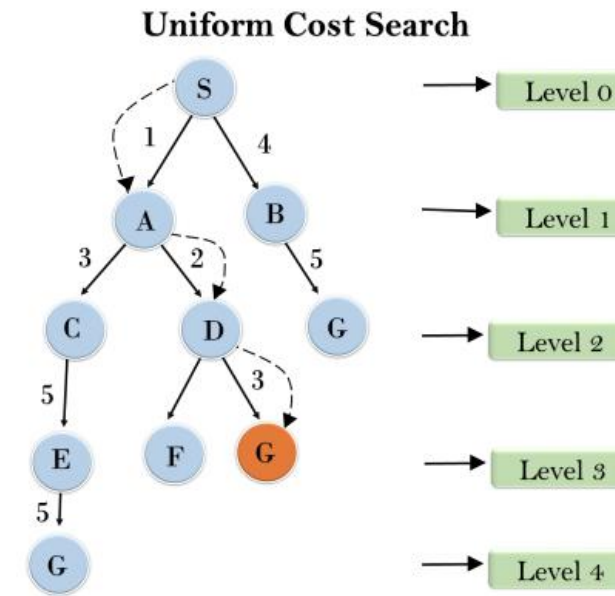
- Example:



Part of the Romania state space, selected to illustrate uniform-cost search.

Uniform-cost Search Algorithm

- **Advantages:** Uniform cost search is optimal because at every state the path with the least cost is chosen.
- **Disadvantages:** It does not care about the number of steps involved in searching and only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.

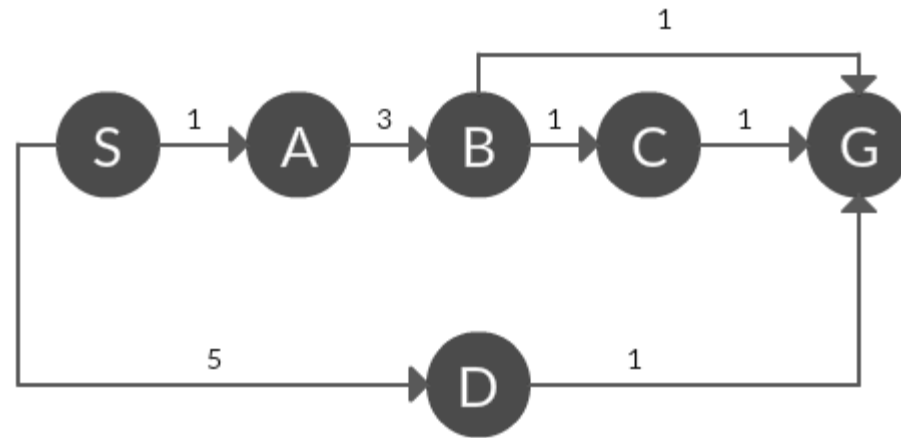


Uniform-cost Search Algorithm

- **Completeness:** Uniform-cost search is complete, such as if there is a solution, UCS will find it.
- **Time Complexity:** Let C^* is Cost of the optimal solution, and ϵ (a lower bound on the cost of each action) is each step to get closer to the goal node. Then the number of steps is $= C^*/\epsilon + 1$. Here we have taken +1, as we start from state 0 and end to C^*/ϵ .
 - Hence, the worst-case time complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.
- **Space Complexity:** The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.
- **Optimal:** Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

Uniform-cost Search Algorithm

- Which solution would UCS find to move from node S to node G if run on the graph below?



Uniform-cost Search Algorithm

- Algorithm:

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution node, or *failure*
return BEST-FIRST-SEARCH(*problem*, PATH-COST)

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*
 node $\leftarrow$ NODE(STATE=*problem*.INITIAL)
 frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element
 reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*
 while not IS-EMPTY(*frontier*) **do**
 node $\leftarrow$ POP(*frontier*)
 if *problem*.IS-GOAL(*node*.STATE) **then return** *node*
 for each *child* **in** EXPAND(*problem*, *node*) **do**
 s $\leftarrow$ *child*.STATE
 if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**
 reached[*s*] $\leftarrow$ *child*
 add *child* to *frontier*
 return *failure*

function EXPAND(*problem*, *node*) **yields** nodes
 s $\leftarrow$ *node*.STATE
 for each *action* **in** *problem*.ACTIONS(*s*) **do**
 s' $\leftarrow$ *problem*.RESULT(*s*, *action*)
 cost $\leftarrow$ *node*.PATH-COST + *problem*.ACTION-COST(*s*, *action*, *s'*)
 yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

Depth-first search

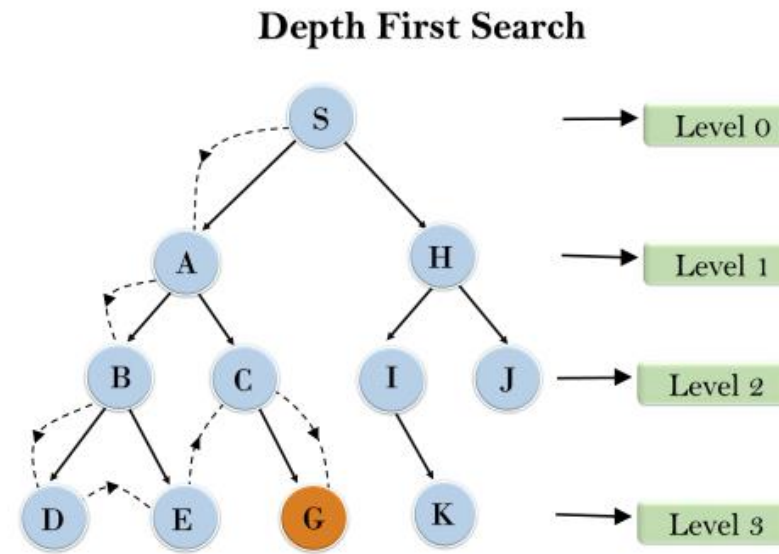
- Depth-first search is a **recursive algorithm** for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and **follows each path to its greatest depth node before moving to the next path**.
- DFS uses a **stack data structure** for its implementation.

Depth-first search

- The search proceeds immediately to the deepest level of the search tree, where the nodes have no successors.
- The search then “backs up” to the next deepest node that still has unexpanded successors.
- Depth-first search is not cost-optimal; it returns the first solution it finds, even if it is not cheapest.

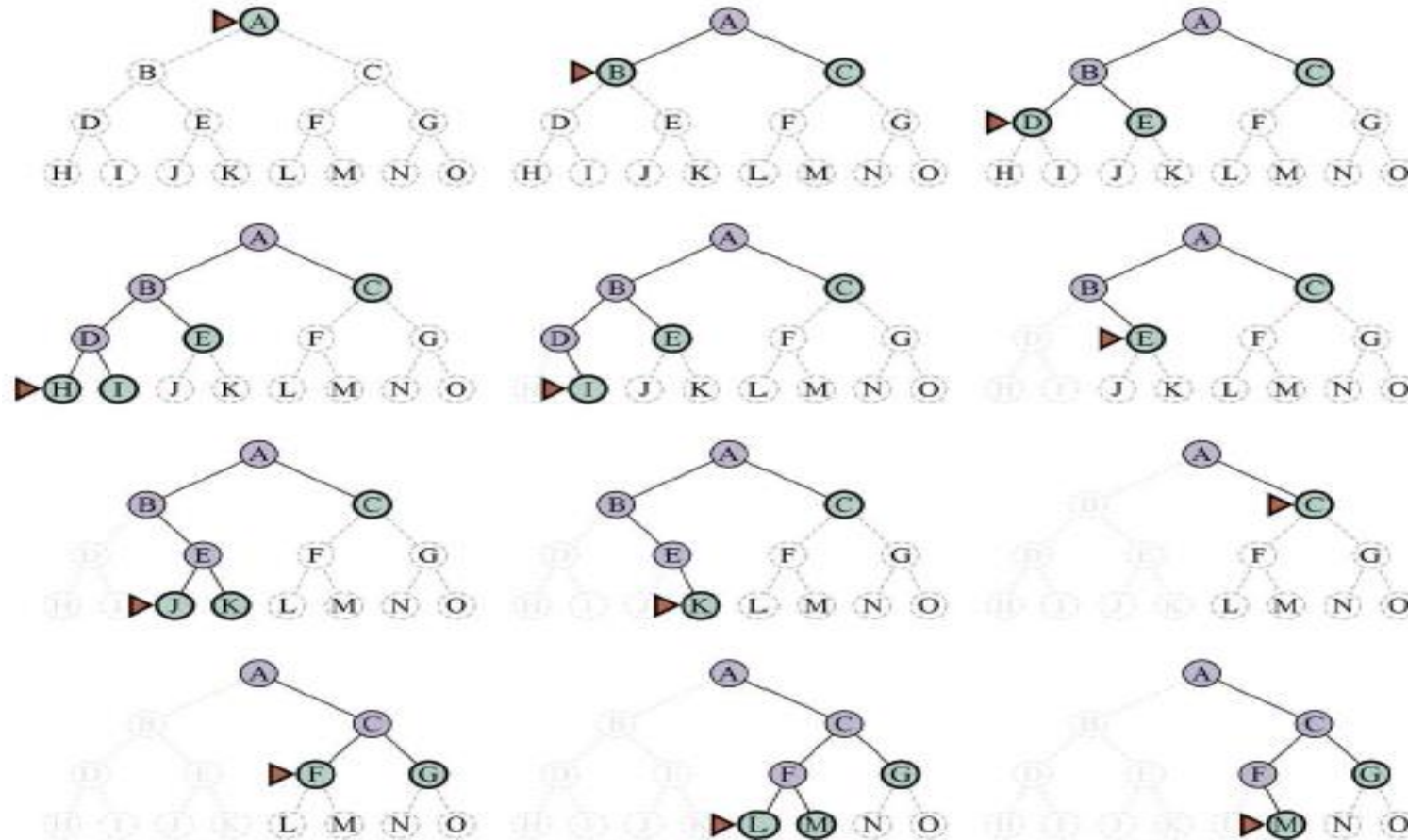
Depth-first search

- Root node--->Left node ----> right node.
- It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.



Depth-first search

- Example:



A dozen steps (left to right, top to bottom) in the progress of a depth-first search on a binary tree from start state A to goal M. The frontier is in green, with a triangle marking the node to be expanded next. Previously expanded nodes are lavender, and potential future nodes have faint dashed lines. Expanded nodes with no descendants in the frontier (very faint lines) can be discarded.

Depth-first search

- Complete? No: fails in infinite-depth spaces, spaces with loops
 - Modify to avoid repeated states along path
 - - complete in finite spaces
- Time? $O(b^m)$: terrible if m is much larger than d
 - but if solutions are dense, may be much faster than breadth-first
- Space? $O(bm)$, i.e., linear space!
- Optimal? No

Depth-limited search

- A depth-limited search algorithm is similar to **depth-first search with a predetermined limit**.

depth-first search with depth limit l

- Depth-limited search can solve the drawback of the **infinite path in the Depth-first search**. In this algorithm, the node at the **depth limit will treat as it has no successor nodes further**.
- Depth-limited search can be terminated with **two conditions of failure**:
 - **Standard failure value**: It indicates that **problem does not have any solution**.
 - **Cutoff failure value**: It defines no solution for the problem within a given depth limit.
- **Advantages**:
 - Depth-limited search is **Memory efficient**.
- **Disadvantages**:
 - Depth-limited search also has a disadvantage **of incompleteness**.
 - It may **not be optimal** if the problem has more than one solution.

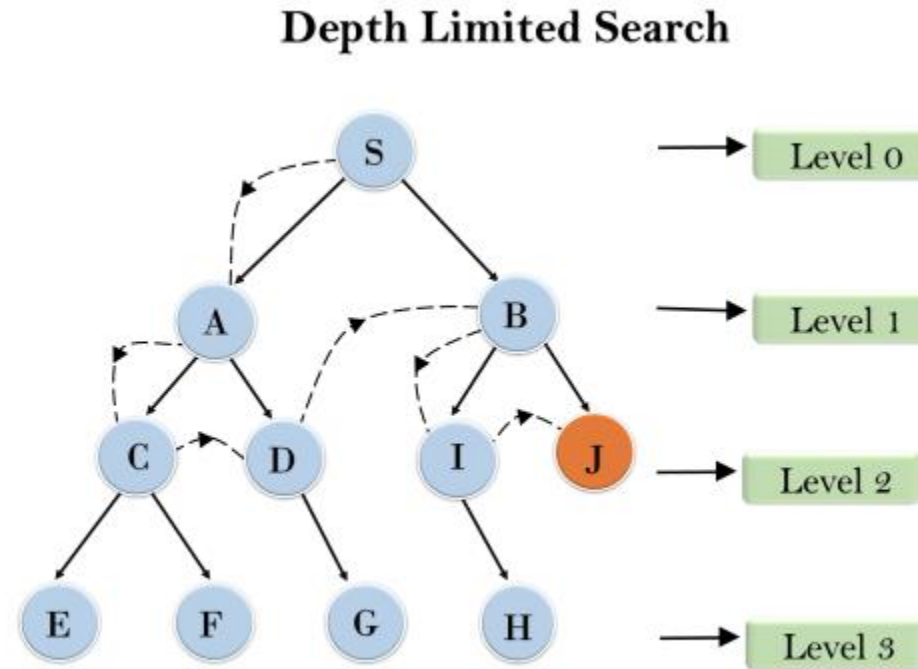
Algorithm

This algorithm essentially follows a similar set of steps as in the DFS algorithm.

1. The start node or node 1 is added to the beginning of the stack.
2. Then it is marked as visited, and if node 1 is not the goal node in the search, then we push second node 2 on top of the stack.
3. Next, we mark it as visited and check if node 2 is the goal node or not.
4. If node 2 is not found to be the goal node, then we push node 4 on top of the stack.
5. Now we search in the same depth limit and move along depth-wise to check for the goal nodes.
6. If Node 4 is also not found to be the goal node and **depth limit is found to be reached**, then we **retrace back to nearest nodes that remain unvisited or unexplored**.
7. Then we push them into the stack and mark them visited.
8. We continue to perform these steps in iterative ways unless the goal node is reached or until all nodes within depth limit have been explored for the goal.

Depth-limited search

- Example: depth limit 2



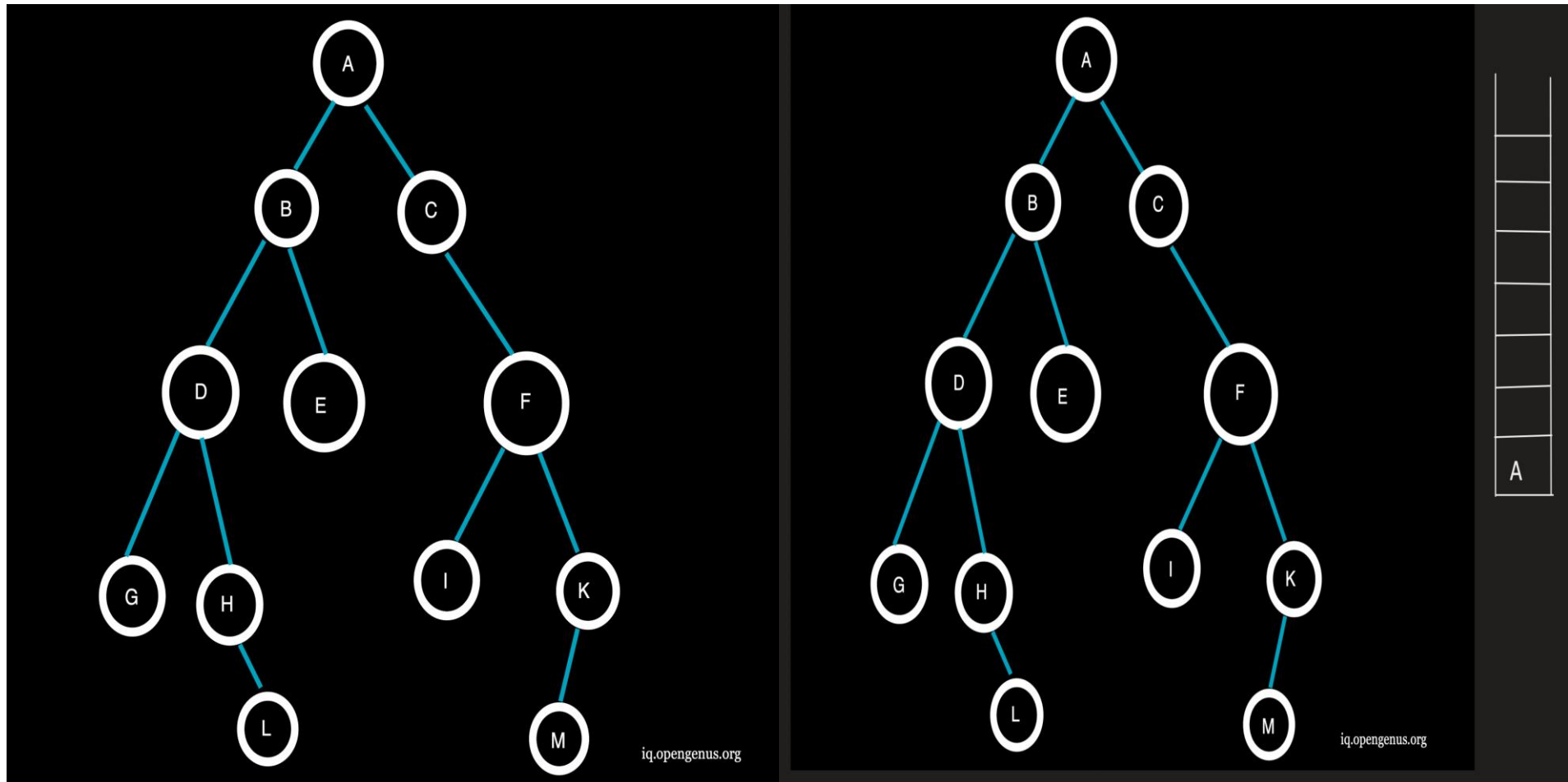
Depth-limited search

- **Completeness:** DLS search algorithm is complete if the solution is above the depth-limit.
- **Time Complexity:** $O(b^\ell)$.
- **Space Complexity:** $O(b \times \ell)$.
- **Optimal:** Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $\ell > d$

Depth-limited search

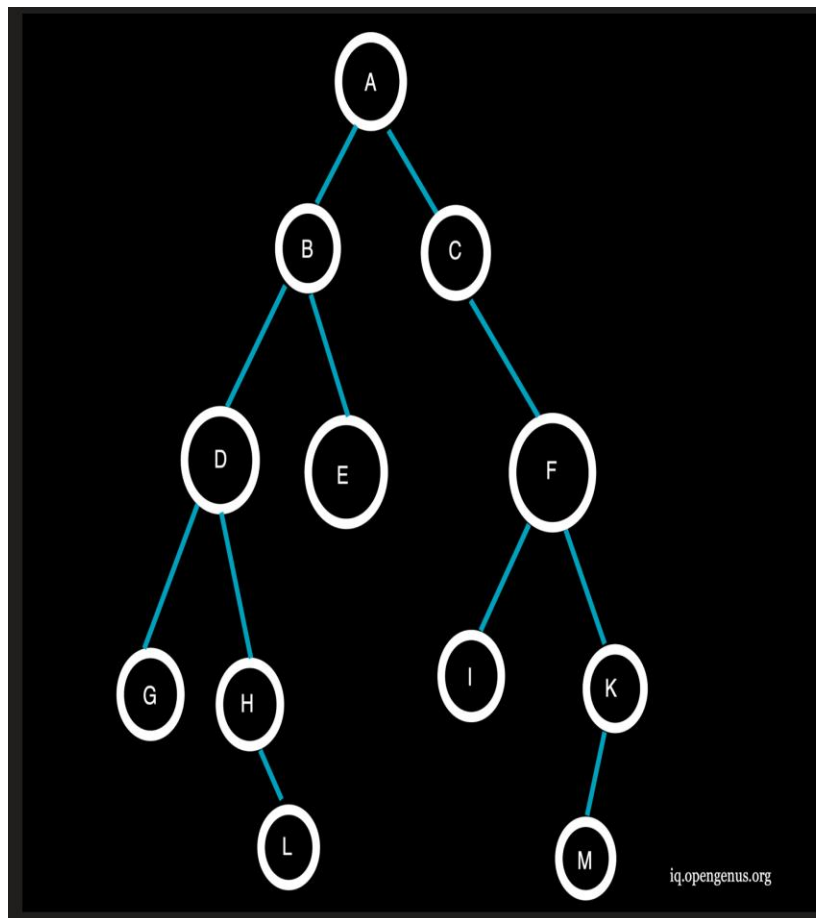
- Algorithm:

```
function DEPTH-LIMITED-SEARCH(problem,  $\ell$ ) returns a node or failure  
  frontier  $\leftarrow$  a LIFO queue (stack) with NODE(problem.INITIAL) as an element  
  result  $\leftarrow$  failure  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    if DEPTH(node)  $\geq \ell$  then  
      result  $\leftarrow$  cutoff  
  
    else if not IS-CYCLE(node) do  
      for each child in EXPAND(problem, node) do  
        add child to frontier  
  return result
```



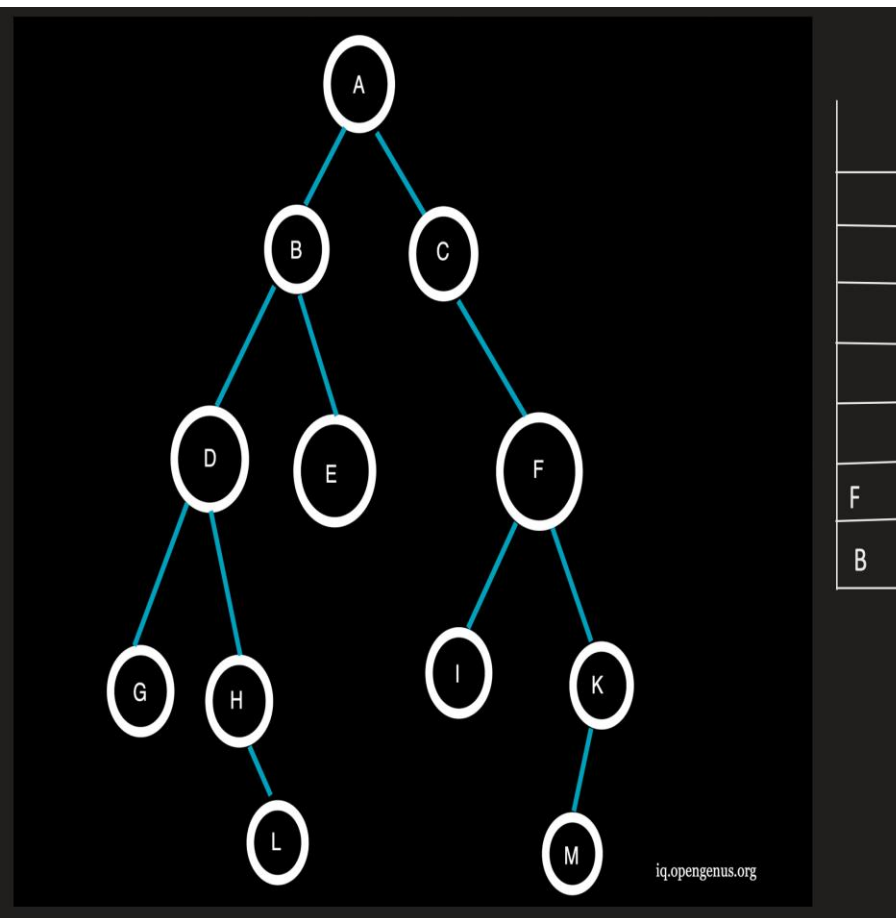
Consider the given graph with Depth Limit(l)=2, Target Node=H and the given source node=A

Now, the first element of the source node is pushed onto the stack.



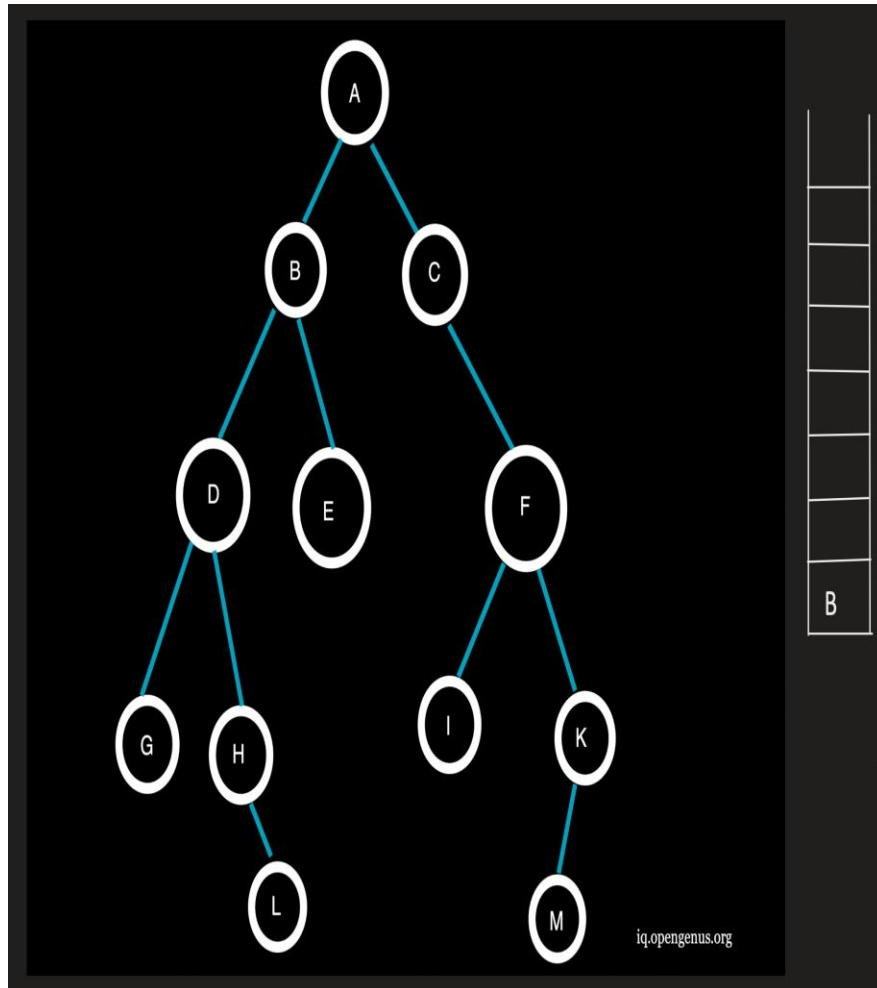
A being the top element is now popped from the stack and the neighbouring nodes B and C at depth=1($< \infty$) of A are pushed onto the stack.

Traversal: A

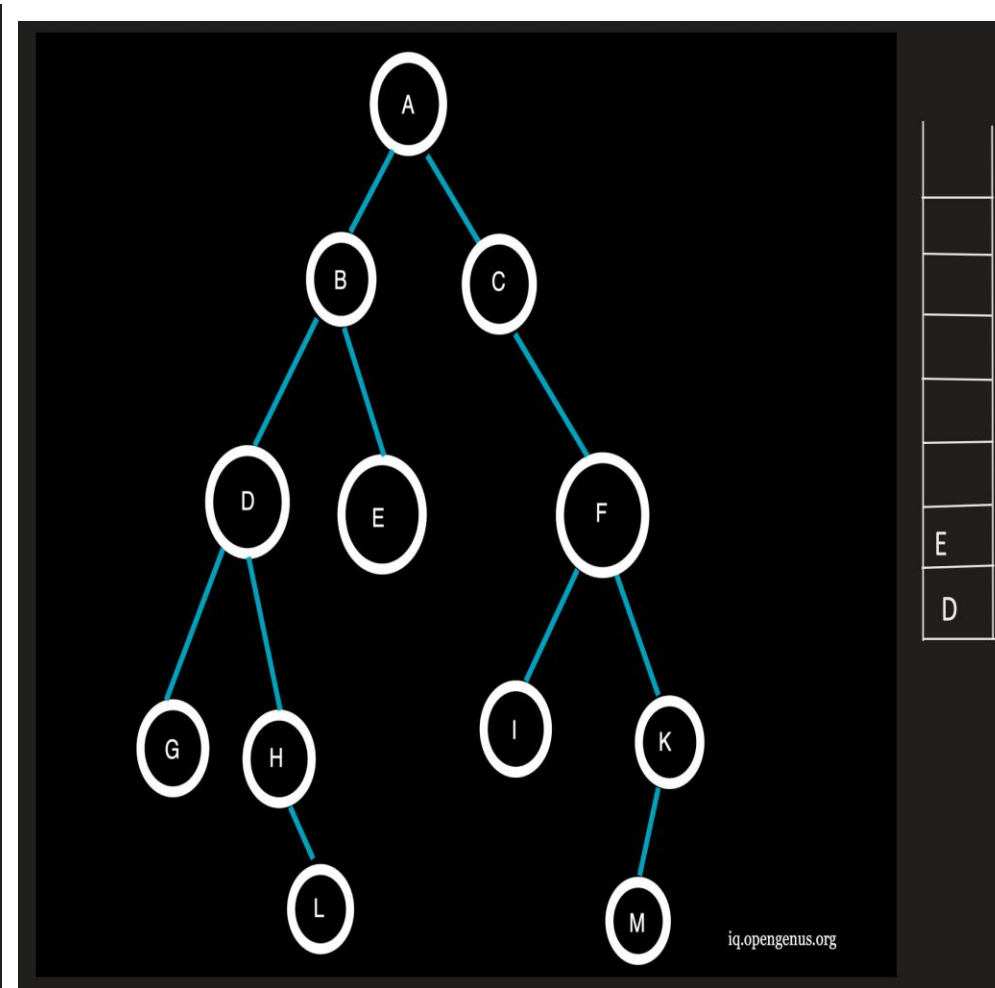


C being the topmost element is popped from the stack and the neighbouring node F at depth=2($= \infty$) is pushed onto the stack.

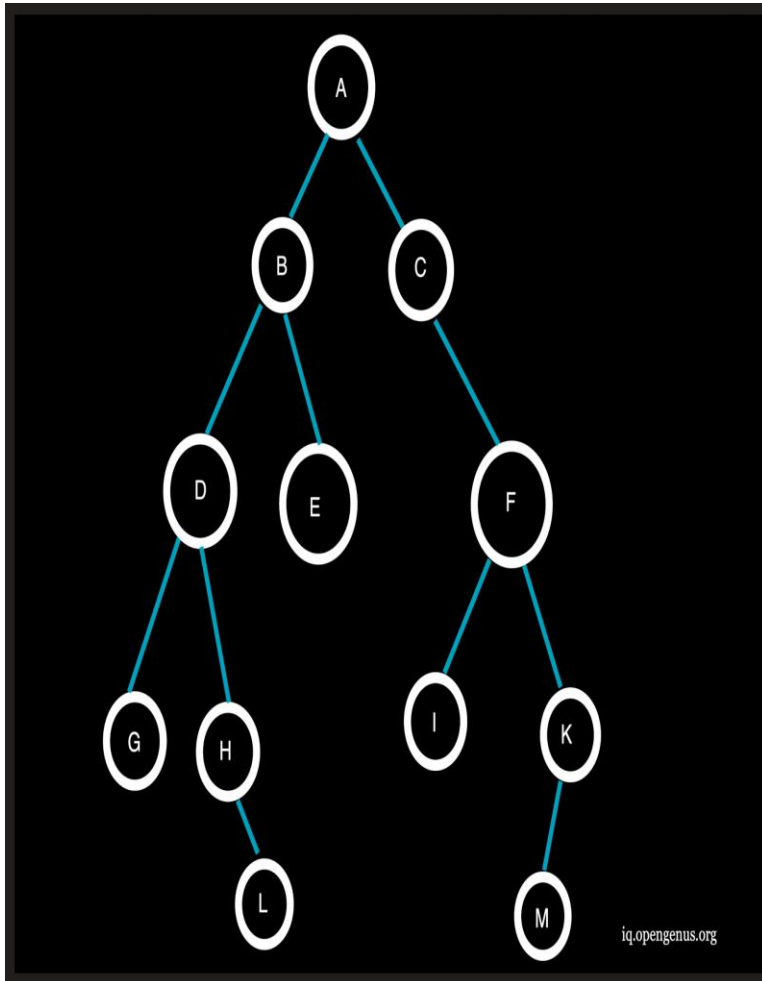
Traversal: AC



F being the topmost element is popped from the stack and the neighbouring nodes I and J at depth=3(>1) will not be pushed onto the stack.
 Traversal: ACF

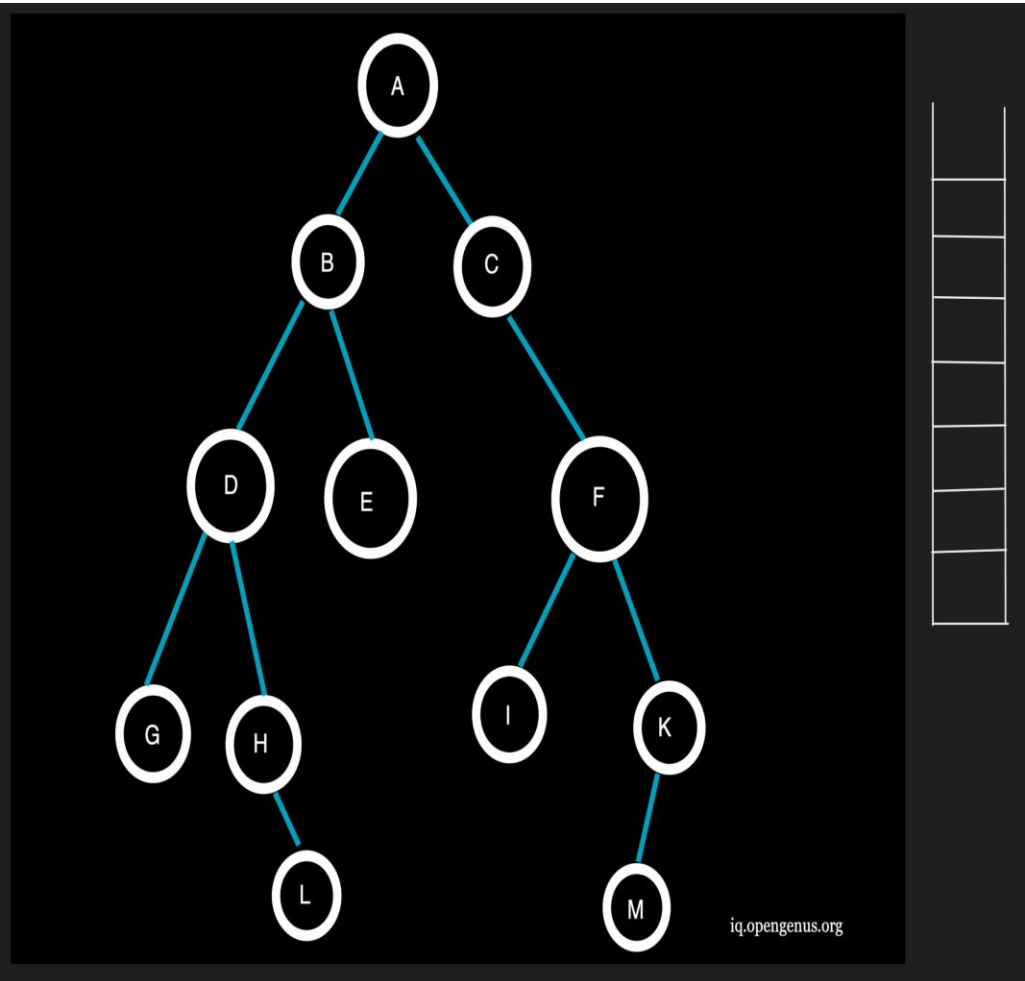


B being the topmost element is popped from the stack and the neighbouring nodes D and E at depth=2($=1$) are pushed onto the stack.
 Traversal: ACFB



E being the topmost element is popped from the stack and since E has no neighbouring nodes, no nodes are pushed onto the stack.

Traversal: ACFBE



D being the topmost element is popped from the stack and the neighbouring nodes G and H at depth=3(>1) will not be pushed onto the stack.

Traversal: ACFBED

Since the stack is now empty, all nodes within the depth limit have been visited, but the target node H has not been reached.

Iterative deepening depth-first Search

- The iterative deepening algorithm is a combination of DFS and BFS algorithms.
- This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.
- This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.
- This search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency.
- The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

111
* iterative deeping search

↳ combination of BFS + DFS

Adv. BFS

(i) [it will find Goal Node at any way.]

dis. BFS

⇒ it consumes very much memo.

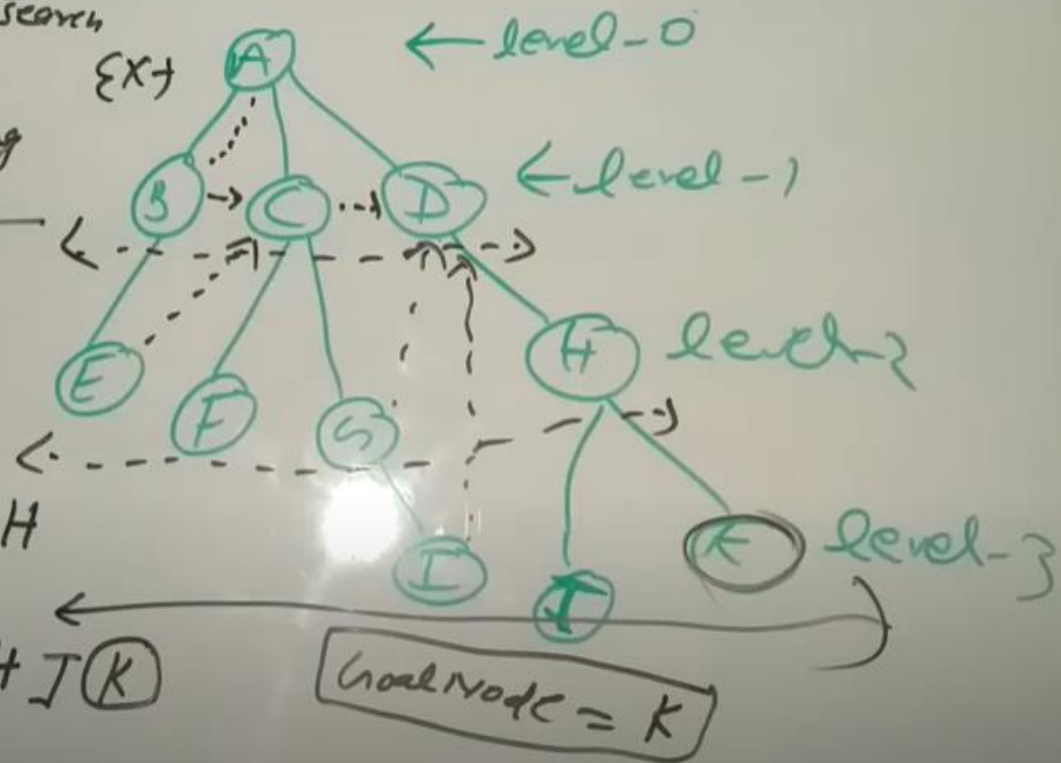
Adv. DFS

[⇒ it consume very less memory.]

* Iterative Depth Search

Depth or level | Iterative deepening search

0	A
1	A B C D
2	A B E C F G H
3	A B E C F G I D H J (K)



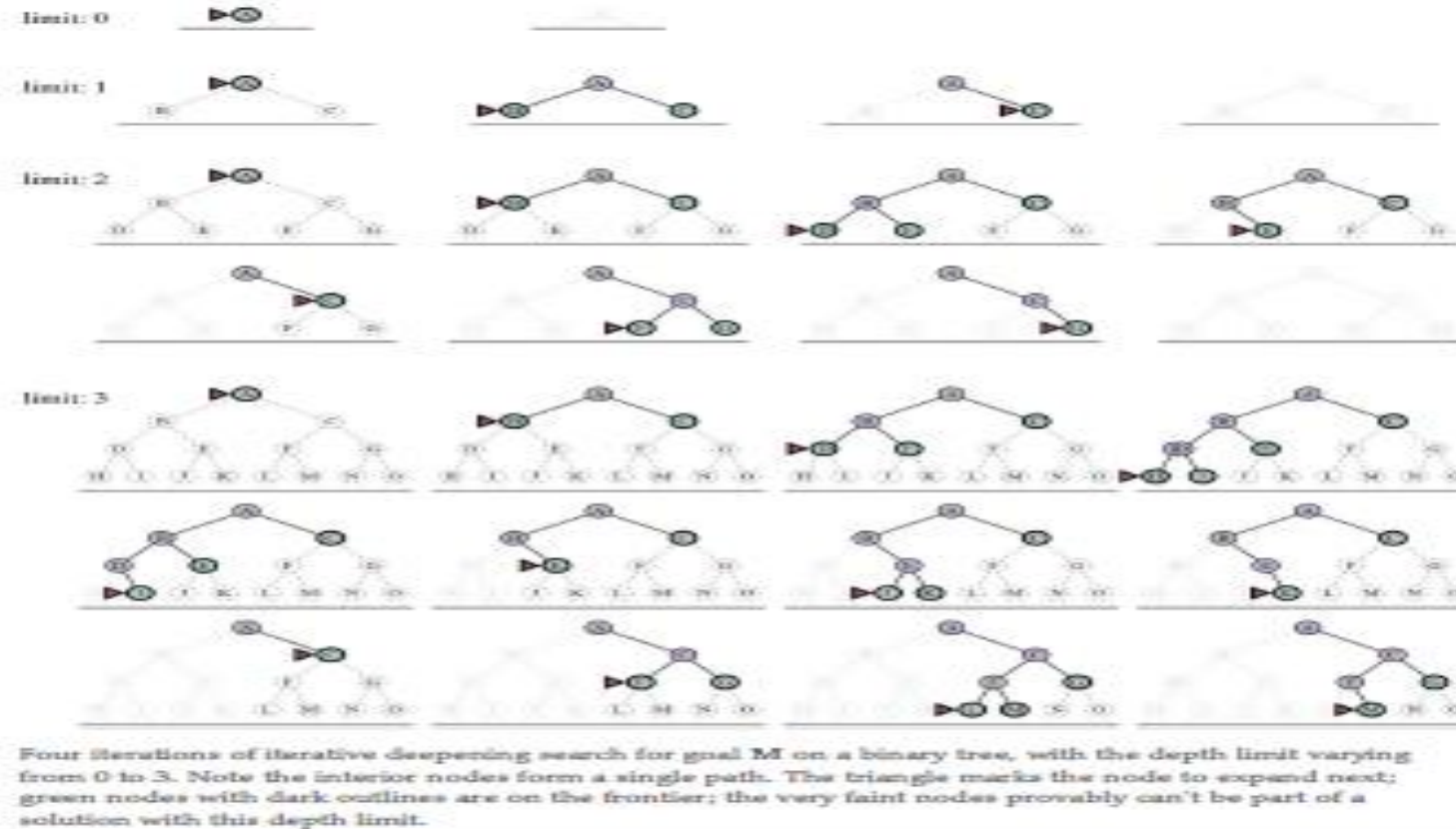
Iterative deepening depth-first Search

- Algorithm:

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure
  for depth = 0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result

function DEPTH-LIMITED-SEARCH(problem,  $\ell$ ) returns a node or failure or cutoff
  frontier  $\leftarrow$  a LIFO queue (stack) with NODE(problem.INITIAL) as an element
  result  $\leftarrow$  failure
  while not IS-EMPTY(frontier) do
    node  $\leftarrow$  POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    if DEPTH(node) >  $\ell$  then
      result  $\leftarrow$  cutoff
    else if not IS-CYCLE(node) do
      for each child in EXPAND(problem, node) do
        add child to frontier
  return result
```

Iterative deepening depth-first Search



Iterative deepening search may seem wasteful because states near the top of the search tree are re-generated multiple times. But for many state spaces, most of the nodes are in the bottom level, so it does not matter much that the upper levels are repeated. In an iterative deepening search, the nodes on the bottom level (depth d) are generated once, those on the next-to-bottom level are generated twice, and so on, up to the children of the root, which are generated d times. So the total number of nodes generated in the worst case is

Iterative deepening depth-first Search

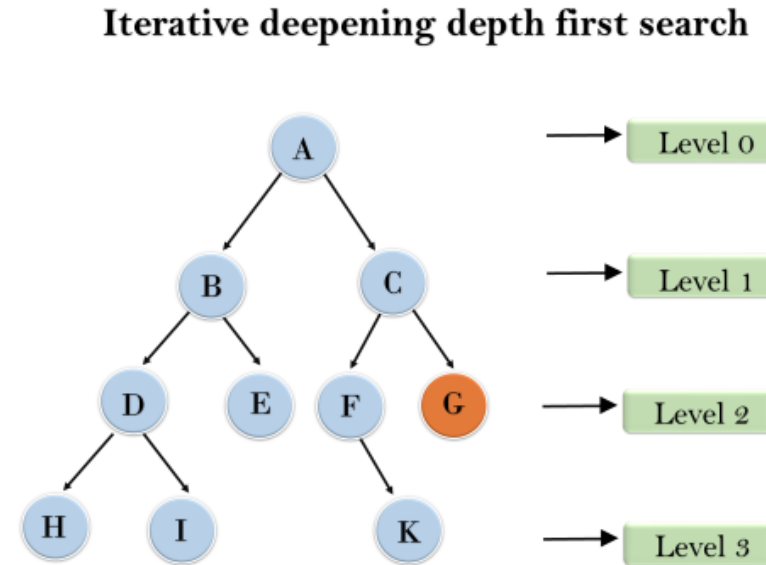
- Complete? Yes
- Time? $O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step cost = 1

Iterative deepening depth-first Search

- Complete? Yes
- Time? $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d \Rightarrow O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step cost = 1

Iterative deepening depth-first Search

- Example:
- 1'st Iteration-----> A
- 2'nd Iteration-----> A, B, C
- 3'rd Iteration----->A, B, D, E, C, F, G
- 4'th Iteration----->A, B, D, H, I, E, C, F, K, G



- **Advantages**

- DS is superior to other search algorithms in a number of ways. The first benefit is that it is comprehensive, **which ensures that a solution will be found if one is there in the search space**. This is so that all nodes under a specific depth limit are investigated before the depth limit is raised by IDS, which does a depth-limited DFS.
- **IDS is memory-efficient**, which is its second benefit. This is because IDS decreases the algorithm's memory needs by not storing every node in the search area in memory. IDS minimises the algorithm's memory footprint by only storing the nodes up to the current depth limit.
- **IDS's ability to be utilised for both tree and graph search is its third benefit**. This is due to the fact that IDS is a generic search algorithm that works on any search space, including a tree or a graph.

- **Disadvantages**

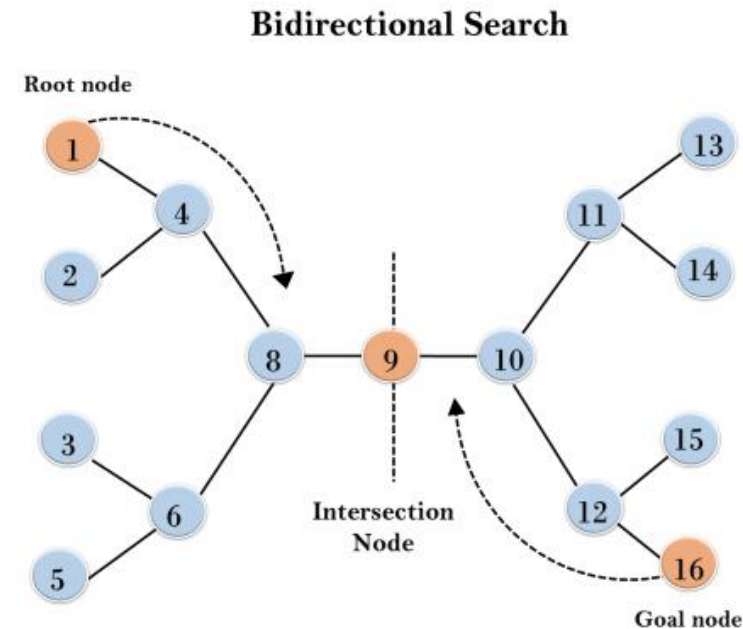
- IDS has the disadvantage of potentially **visiting certain nodes more than once**, which might slow down the search. The benefits of completeness and optimality frequently exceed this disadvantage. In addition, by employing strategies like memory or caching, the **repeated trips can be minimised**.

Bidirectional Search Algorithm

- Bidirectional search algorithm runs **two simultaneous searches**, one from initial state called as **forward-search** and other from goal node called as **backward-search**, to find the goal node.
- Bidirectional search replaces **one single search graph** with **two small subgraphs** in which one starts the search from an **initial vertex** and other starts from **goal vertex**. The search stops when these **two graphs intersect** each other.
- Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.
- **Advantages:**
 - Bidirectional search is fast.
 - Bidirectional search requires less memory
- **Disadvantages:**
 - Implementation of the bidirectional search tree is difficult.

Bidirectional Search Algorithm

- In the below search tree, bidirectional search algorithm is applied.
- This algorithm divides **one graph/tree into two sub-graphs**.
- It starts traversing from node 1 in the **forward direction** and starts from goal node 16 in the **backward direction**.
- The algorithm terminates at node 9 where two searches meet.



Bidirectional Search Algorithm

- **Completeness:** Bidirectional Search is complete if we use BFS in both searches.
- **Time Complexity:** $O(b^{d/2})$.
- **Space Complexity:** $O(b^{d/2})$.
- **Optimal:** Bidirectional search is Optimal.

Comparing uninformed search algorithms

- Summary

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Evaluation of search algorithms. b is the branching factor; m is the maximum depth of the search tree; d is the depth of the shallowest solution, or is m when there is no solution; ℓ is the depth limit. Superscript caveats are as follows: ¹ complete if b is finite, and the state space either has a solution or is finite. ² complete if all action costs are $\geq \epsilon > 0$; ³ cost-optimal if action costs are all identical; ⁴ if both directions are breadth-first or uniform-cost.